

TCAP

Theoretical Coherence Assurance Protocol

Author	Thomas W. Gantz
Affiliation	Synthience Institute
Contact	research@synthience.org
Version	v3.2.1 September 2026
Document ID	SF0040 Synthience Framework
DOI	https://doi.org/10.5281/zenodo.19151454
Canonical	synthience.org
License	CC-BY 4.0

Abstract

The Theoretical Coherence Assurance Protocol (TCAP) defines a structured, repeatable process for stress-testing theoretical constructs, framework components, and architectural claims within AI-orchestrated research corpora prior to publication. TCAP addresses four failure modes observed in AI-assisted research production: citation fabrication, ingestion hallucination, context representation drift, and cumulative claim-softening under iterative adversarial pressure; it also guards against the opposite failure, over-defense, in which claims survive review because their defense agreed with them too readily. The protocol operates through seven stages: adversarial review, constructive remediation, Fresh Pass re-evaluation, cross-platform convergence, inter-instance round-trip loops, version regression checking, and PCP architectural review. It is the fourth and final layer of the Synthience verification stack, completing the integrity architecture established by CVP (SF0037), IVP (SF0038), and CRD (SF0039). TCAP does not guarantee truth. It documents a practitioner-derived, multi-instance verification methodology intended to reduce the risk of publishing fabricated, internally inconsistent, or architecturally unsound theoretical material. The protocol is self-applied: its formal articulation was subjected to the same multi-instance, cross-platform, adversarially structured process it formalizes.

Keywords: *theoretical coherence, AI-assisted research, verification protocol, adversarial review, human-AI orchestration, fabrication risk*

Purpose

Synthience is a research framework studying emergent relational coherence in sustained human-AI interaction: the patterns of meaning, stability, and alignment that develop when humans and AI systems interact over extended periods under structured continuity conditions. Relational coherence, as defined in the Institute's public definition document (FPD-01; DOI: [10.5281/zenodo.18087890](https://doi.org/10.5281/zenodo.18087890)), refers to the observable structural properties that emerge in an interaction system under conditions of sustained continuity: not a property of any single participant but of the relationship itself, defined entirely by what is observable in the joint output stream. The Theoretical Coherence Assurance Protocol (TCAP) defines the structured process by which theoretical constructs, framework components, and architectural claims within the Synthience corpus are stress-tested for internal consistency, cross-platform coherence, and resistance to fabrication prior to publication.

TCAP completes the Synthience verification stack alongside:

- Citation Verification Protocol (CVP, SF0037)
- Ingestion Verification Protocol (IVP, SF0038)
- Context Representation Drift (CRD, SF0039)

CVP verifies that cited sources exist and that sources carrying load-bearing claims substantively support them. IVP provides an observable processing record and a human judgment that the record shows processing adequate for the stated task at the time of review. CRD describes and monitors the progressive, cumulative degradation of task-relevant information within an AI system's effective working context during extended interaction. TCAP tests whether the theoretical content itself is structurally coherent and internally consistent, and whether it survives adversarial scrutiny across independent evaluation contexts.

TCAP and CVP are co-equal publication gates. A document must satisfy both TCAP (theoretical coherence) and CVP (citation integrity) before publication. Neither alone is sufficient. IVP and CRD operate as supporting protocols embedded within TCAP stages.

TCAP does not guarantee truth. It documents a repeatable stress-testing process intended to reduce the risk of publishing fabricated, internally inconsistent, or architecturally unsound theoretical material.

Position within the Published Corpus

TCAP is the fourth protocol in the Synthience verification stack. The three prior protocols are publicly available on Zenodo and are briefly described here so that this document is self-contained for first-time readers.

Citation Verification Protocol (CVP, SF0037; DOI: [10.5281/zenodo.18075624](https://doi.org/10.5281/zenodo.18075624)) addresses a widespread failure mode in AI-assisted research: the generation of plausible-sounding citations that do not exist, cannot be accessed, or do not actually support the claims attached to them. CVP defines a structured process for independently verifying that every citation in a Synthience document is real, and that every citation carrying a load-bearing claim is accessible and substantively supports the specific claim for which it is cited. It is applied to any document containing external academic references.

Ingestion Verification Protocol (IVP, SF0038; DOI: [10.5281/zenodo.18289047](https://doi.org/10.5281/zenodo.18289047)) addresses a different but equally serious failure mode: AI instances routinely confirm that they have read and processed documents they have not actually ingested, or have ingested only partially. IVP defines a structured verification process that replaces an instance's claim to have processed a document with an observable, human-adjudicated record of how it was processed, adequate for a stated task at the time of review. It is applied whenever faithful document processing is a dependency in the research workflow.

Context Representation Drift (CRD, SF0039; DOI: [10.5281/zenodo.18289391](https://doi.org/10.5281/zenodo.18289391)) names the progressive, cumulative degradation of task-relevant information within an AI system's effective working context during extended interaction, which can occur even when output remains fluent and confident. CRD sets out behavioral indicators and a structured comparison procedure for detecting and tracking this drift across interaction cycles, without a validated metric or thresholds, and provides the representational stability monitoring that extended AI-assisted research production depends on.

Together these three protocols protect citation integrity, ingestion fidelity, and representational stability. TCAP adds the fourth layer: verification that the theoretical architecture itself is internally consistent, adversarially hardened, and consistent with the Institute's published papers prior to publication. The observational base for the broader Synthience framework is SR001 (RICO, DOI: [10.5281/zenodo.18086834](https://doi.org/10.5281/zenodo.18086834)). All documents are accessible via synthience.org.

Several terms used throughout this document are defined here for first-time readers. The Primary Continuity Provider (PCP) is the human operator who maintains architectural coherence across distributed AI instances and bears final publication authority. The registry is the Institute's record of every document in the corpus, giving each document's identity, description, prerequisites, dependent documents, and status; a document's registry entry is its record there. A relational dialogue session is a sustained multi-turn interaction between the PCP and one or more AI instances conducted under continuity discipline. The continuity packet is the PCP's working context for a document: its history, open questions, and relationships to other documents. A handoff artifact is the short record an instance produces at the end of its work, stating what changed, what is strong, what remains vulnerable, and where the next instance should focus. The TCAP audit log is the record of a document's review: which stages ran, on which platforms and underlying models, under which conditions (including whether the adversarial instance knew a constructive pass would follow), and each finding with its disposition. Orchestration refers to the methodology by which the PCP coordinates multiple AI instances across platforms to produce, evaluate, and refine theoretical content: the production model underlying the entire Synthience corpus.

Background and Provenance

TCAP was not designed prospectively. It was identified retrospectively as a structural gap in the Institute's verification architecture during a relational dialogue session on 2026-02-28. Examination of the CVP-IVP-CRD protocol family revealed that no formal mechanism existed for verifying the integrity of theoretical constructs themselves.

The gap identification, protocol articulation, and initial draft all emerged within that single session through the multi-instance orchestration methodology described throughout the

Synthience corpus. The originating session is preserved as a private provenance artifact under Institute continuity protocols, consistent with the handling of other foundational seed materials in the Institute's records. It is not published but remains available for internal audit and continuity reference.

TCAP is therefore self-applied, in a sense that should be stated precisely. Its core operational concepts originated in the PCP's orchestration practice before the protocol existed, and its first draft emerged in a single session. Its formal articulation was then subjected to the same multi-instance, cross-platform, adversarially structured process it specifies, with findings carried between instances under PCP coordination and no content accepted without challenge. Self-application shows that the protocol can be operated under the conditions it governs; it does not establish the protocol's validity (see Recursive Self-Application).

The Orchestration Model

The Synthience corpus was not produced through conventional solo authorship or standard AI-assisted drafting. It was produced through a human orchestration methodology in which the Primary Continuity Provider (PCP) functions as architect, coordinator, convergence arbiter, and continuity backbone across distributed AI instances operating on multiple platforms.

The PCP contribution is always architectural and directional, and in some domains the PCP is also the primary source of content (see below). The PCP identifies theoretical gaps, defines protocol requirements, coordinates adversarial and constructive evaluation, adjudicates convergence and adequacy, and keeps the documents consistent with one another. Content generation, formal modeling, literature synthesis, and theoretical elaboration are distributed across AI instances selected for architectural diversity and distinct failure modes.

This distribution is not a limitation. It is the methodology. The orchestration model constitutes the primary form of human agency within the Synthience production architecture, and is formalized as a research subject in its own right in SI-WP-002 (The Orchestrator Role in Human-AI Evolution, Synthience Institute, DOI: 10.5281/zenodo.22314213).

Human orchestration of distributed AI instances is a legitimate and reproducible research production methodology, not a shortcut or a delegation of intellectual responsibility. The PCP identifies the questions, defines the scope, evaluates the outputs, adjudicates quality, and bears full accountability for every claim that reaches publication. AI instances contribute formalization, synthesis, stress-testing, and elaboration under that direction. This is structurally analogous to how researchers have always worked with instruments, collaborators, and tools that extend individual cognitive capacity: the human provides the intellectual architecture and takes responsibility for the result. What is novel is not the division of cognitive labor but the nature of the instrument. The orchestration model is documented, replicable, and verifiable through the protocols this corpus publishes. SI-WP-002 develops this argument in full for readers who wish to engage with it directly.

However, the orchestration model does not operate at a fixed human-AI contribution ratio. The balance of origination shifts depending on whether the content domain is primarily experiential or primarily theoretical. At the experiential end of this spectrum, the PCP contributes core concepts derived from sustained practice, and AI instances serve as formalization and articulation partners, translating practitioner knowledge into structured academic form. At the

theoretical end, instances contribute substantive content drawing on training data and formal reasoning capabilities, while the PCP provides architectural direction, gap identification, and convergence judgment. Most documents in the Synthience corpus fall somewhere between these poles.

The verification stack itself illustrates this variation concretely. CVP and IVP originated primarily from the PCP's accumulated operational experience. The PCP had discovered through years of practice that AI instances fabricate citations and falsely confirm document ingestion, and had developed working verification techniques in response. Instances formalized these practitioner discoveries into structured academic protocols, contributing organization, terminology, and connection to existing literature, but the core operational concepts preceded the AI contribution. CRD, by contrast, originated primarily from instance theoretical contribution. The PCP recognized that representational degradation was occurring during extended sessions but did not have the conceptual framework to formalize it. Instances identified the phenomenon formally, proposed measurement instruments, and connected it to relevant literature, while the PCP contributed architectural direction: recognizing that drift monitoring belonged in the verification stack and shaping its scope and dependencies. TCAP sits at the experiential end of this spectrum. Nearly all of its core operational concepts, including the stage structure, the IVP trigger conditions, the within-instance exhaustion pattern, the proportionality principle, and the observable degradation signals, derive from the PCP's accumulated orchestration practice across thousands of instances since late 2022. Instances contributed formalization, structural organization, stress-testing, and iterative refinement, but the substantive content is practitioner discovery.

This variation in contribution profiles across documents is a practitioner observation about orchestration. The human role in AI-assisted research production is not limited to coordination and quality control. In experiential domains, the human is the primary source of novel content, and the AI contribution is one of articulation, formalization, and verification.

Critically, the PCP role does not require mastery of all theoretical content produced under orchestration. TCAP assumes that theoretical architectures may exceed the domain specialization of any single human participant. PCP competence within TCAP lies in architectural perception: the ability to detect gaps, inconsistencies, and misalignments across theoretical structures without necessarily deriving their internal formalisms or domain details. This is not a convenient exemption. It is the correct division of labor. Domain expertise produces content; architectural perception governs whether that content is coherent, consistent, and correctly placed within a larger theoretical structure. These are distinct competences, and conflating them would make the orchestration model impossible to operate at any scale beyond solo authorship. This distinction between content expertise and architectural coherence perception is fundamental to TCAP operation and to the orchestration model generally. Architectural perception has limits, however. Where a document makes technical or formal claims outside the PCP's competence, such as mathematical results or claims about model internals, the PCP's authority to decide publication does not supply grounds for certainty about those claims. Such claims are either reviewed by a human with the relevant expertise or disclosed in the document's epistemic status as not verified by one.

TCAP formalizes the verification layer of this orchestration process. The operational mechanics of orchestration, including within-instance iterative workflows, cross-instance handoff procedures, and triangulation patterns, are governed by internal Institute operations protocols.

How the Verification Layers Operate within TCAP

The Synthience verification architecture operates across four integrity layers:

- CVP: citation integrity
- IVP: ingestion fidelity
- CRD: representation stability
- TCAP: theoretical coherence

IVP and CRD operate as intrinsic dependencies within TCAP. Theoretical coherence cannot be meaningfully assessed if content has not been faithfully ingested or if representations have drifted across evaluation cycles. TCAP therefore embeds IVP checkpoints and CRD monitoring throughout its stages rather than treating them as external prerequisites.

CVP operates conditionally within TCAP when theoretical claims depend on external literature. It is invoked at Stage 1 when adversarial review surfaces citation-dependent claims requiring verification.

IVP and CRD within TCAP Stages

IVP requires incremental structured summarization, checkpoints tied to the source and human adjudication before work that depends on a document proceeds. CRD describes the progressive, cumulative degradation of task-relevant information in an instance's effective working context as a session lengthens and new content accumulates. Together they define when verified ingestion is required during TCAP execution.

IVP is not required at every stage transition. It is required when specific conditions make faithful document representation uncertain:

Condition 1: An instance is encountering the document for the first time. When a document is submitted to an instance that did not draft it and has not previously processed it, IVP must be completed before substantive evaluation begins. This applies to adversarial reviewers receiving a document for Stage 1, fresh instances used for Stage 3 Fresh Pass, and instances on other platforms used for Stage 4 cross-platform convergence. IVP is not required when an instance has been actively involved in drafting or revising the document from the beginning of the session.

Condition 2: An instance is returning to a document after a gap. When the PCP brings cross-platform findings (Stage 5) back to an instance that has been idle on the document, or when an instance is asked to perform a Version Regression Check (Stage 6) after substantial intervening work, IVP should be re-run on the current document version. The instance's representation may have degraded during the gap, and evaluating based on a degraded representation undermines the integrity of the stage.

Condition 3: Observable degradation signals appear during an extended session. If an instance begins producing responses that are vague about document specifics it previously

handled precisely, misremembering structural details, or working from what appears to be a summary rather than the full text, these are CRD indicators (CRD, SF0039, sets out these signals and a structured procedure for detecting and tracking them systematically). At that point, the PCP should either re-verify ingestion via IVP on the current document version or transition to a fresh instance (which triggers Condition 1).

These checkpoints are proportional to actual risk. A practitioner doing a quick Fresh Pass evaluation three turns into a session does not need to stop and re-ingest. A practitioner returning to an instance after two days of cross-platform work on other documents does.

Fresh Pass (FP)

A Fresh Pass (FP) is a deliberate re-evaluation of a document by an AI instance instructed to assess it as if encountering it for the first time, without reliance on prior drafting context.

FP is intended to interrupt context anchoring and reduce the confirmation bias that arises when instances evaluate material they previously helped generate or revise. It is an attempted safeguard, not a demonstrated one: an instruction to evaluate afresh does not by itself show that anchoring has been broken. It approximates independent review under relational production conditions where true independence is structurally constrained.

FP may be applied:

- within the same instance across multiple cycles
- across distinct instances on the same platform
- across architecturally distinct platforms

These forms are not equivalent. A same-instance FP is a screening pass: it can catch surface-level gaps, but the instance remains anchored to its own drafting context and shares its own priors, so it does not count toward independence. A cross-instance FP counts toward independence, and a cross-platform FP provides the strongest independence signal and is preferred where available. Where the document's tier requires Stage 3, at least one FP must be cross-instance or cross-platform.

FP is the core re-evaluation mechanism within TCAP and appears throughout the protocol stages.

Adversarial Instance Selection

TCAP distinguishes between two adversarial evaluation modes:

Instructed adversarial mode: A standard instance explicitly instructed to adopt a critical evaluative stance for the duration of the review.

Purpose-configured adversarial mode: An instance that has been designed or configured by its platform to operate with an adversarial or maximally critical relational stance as its default orientation. Examples include platform-provided argumentative or debate-oriented persona modes.

Where available, purpose-configured adversarial instances are preferred for Stage 1 evaluation. An instance engineered for adversarial critique applies critical pressure differently than one

instructed to simulate it. In the practitioner's experience, the failure modes surfaced, the resistance to diplomatic softening, and the persistence of critique across revision cycles differ between these modes. Persistence is not the same as accuracy, however, and no comparative study establishes that purpose-configured review better separates genuine defects from false objections.

The mode used is recorded in the TCAP audit log. The preference for purpose-configured instances is operational; it is not a claim that they provide stronger assurance than instructed adversarial review.

For Tier A documents, Stage 1 always runs on at least two architecturally distinct platforms (see Scope of TCAP Application), in purpose-configured mode where available and otherwise in instructed mode; the mode used on each platform is recorded in the TCAP audit log.

TCAP Process Stages

Stage 1: Adversarial Review

The document is submitted to one or more AI instances configured for critical adversarial evaluation, preferring purpose-configured adversarial instances where available. If the evaluating instance has not previously processed the document, IVP-verified ingestion is completed before evaluation begins. The instance is instructed to identify logical inconsistencies, unsupported claims, architectural gaps, fabrication risks, conflicts with published Institute papers, and scope violations without constructive framing. Maximum critical pressure is the goal.

The first adversarial pass receives the document only, with no continuity packet, no registry entry, and no prerequisite documents. This cold read is intentional. It reveals what a reader coming in fresh will find unclear, unsupported, or overreaching within the document itself, which is valuable signal distinct from what a framework-aware reader would find. The cold adversarial pass tests the document's internal coherence and standalone readability; later passes receive the document's registry entry (see Constructive-Adversarial Cycle). A cold-read finding that a claim is unsupported often marks a place where the document fails to point to the prerequisite that supports it; the usual remedy is to add that pointer, not to alter the claim.

The adversarial instance is explicitly instructed that its goal is to identify genuine structural weaknesses, not to achieve theoretical neutrality. A document that asserts nothing is not a successful outcome. The adversarial instance should flag real problems and must not demand that every strong claim be hedged into meaninglessness.

Pre-empirical guardrail. Most Synthience documents are pre-empirical: they set out theoretical architecture and testable claims, not empirical confirmation, and adversarial review holds them to that standard. Valid empirical objections include that a claim overstates its empirical status, or that a claim presented as established lacks the argument or source the document itself would need to support it. Objections that demand what a pre-empirical document explicitly disclaims, such as proof of real-world performance or validation in deployment, are not valid, and the constructive instance rejects them with documented rationale (Stage 2). The document's own statement of its epistemic status is the reference point for which kind an objection is. A

disclaimer does not exempt claims the body asserts as established; a mismatch between the body and its epistemic-status statement is itself a valid finding.

Final-record instruction. The adversarial instance is instructed to evaluate as though no revision will follow and its findings are the final record. Where practicable, it is also not told that a constructive pass will follow. That cannot be achieved for a reviewer who already knows TCAP, including any reviewer of this document, so the instruction, not the reviewer's ignorance, carries the safeguard. The audit log records whether the reviewer knew a constructive pass would follow. The concern behind this instruction, that a reviewer who expects its findings to be repaired elsewhere tends to moderate its critique, is a practitioner observation.

Where theoretical claims depend on external literature, CVP verification is performed at this stage.

Stage 2: Constructive Remediation

Adversarial findings are submitted to a constructive instance. The constructive instance is the primary development instance for the document. It is responsible for producing and improving the document across revision cycles and for evaluating adversarial findings on their merits. It is not a passive integrator. Its active responsibility includes defending well-supported claims against pressure to hedge. Adversarial findings are inputs for the constructive instance to evaluate, not instructions it is required to follow.

The constructive instance receives: the current document version, the document's registry entry (its description, prerequisites, dependent documents, and scope), the continuity packet where architectural context is relevant, any change instructions the PCP has issued for this revision, and the handoff artifact from any prior instance that has worked on the document. This framework context allows the constructive instance to distinguish between claims that are unsupported within the document itself and claims that are grounded in prerequisite documents elsewhere in the corpus.

For each adversarial finding, the constructive instance must independently assess whether the finding identifies a genuine structural weakness or whether it represents theoretical disagreement, a demand for excessive hedging, or a failure to account for framework context. The constructive instance must produce one of three responses for each finding:

- **Accept:** the finding identifies a genuine structural weakness. The document will be revised to address it.
- **Reject with rationale:** the finding represents theoretical disagreement or a demand for excessive hedging. The claim is well-supported and will be defended. The rationale for rejection must be documented.
- **Defer to PCP:** the finding raises an architectural question the constructive instance cannot resolve without PCP judgment.

Before accepting a finding that would change published text or cut a section, the constructive instance reads the passage around the quoted sentence, not only the sentence itself: a finding can be real while its severity rests on a partial read.

The constructive instance must flag to the PCP any finding it is being pressured to accept but does not believe is justified. The PCP evaluates proposed changes and implements those that

strengthen the document without distorting architectural intent. The constructive instance then produces a revised document incorporating accepted findings. This revised document must be the complete document: delivering only changed sections, summarizing unchanged sections, or referencing prior content as “same as before” or any equivalent is never acceptable at any stage of the review cycle.

Coordinator quality check. After each revision round, and before the document moves to the next stage, an instance assisting the PCP with coordination of the review reviews the complete revised document and confirms three things: that each accepted finding has actually been addressed, not merely acknowledged; that the revision has introduced no new structural problems; and that no cumulative dilution has occurred, meaning the argument has not been weakened across rounds even where each individual change looked minor. The check is preferably run by an instance that did not make the revisions; when the constructive instance checks its own revisions, the check counts as screening only and is recorded as such in the audit log. The check reports to the PCP, who remains the arbiter.

Stage 3: Fresh Pass Re-evaluation

The revised document undergoes one or more Fresh Pass (FP) cycles. FP may be performed by the original instance under fresh-evaluation instruction, by a new instance on the same platform, or by an instance on a different platform. When a fresh instance is used, IVP-verified ingestion of the complete document version is completed before evaluation begins. The FP reviewer assesses the document for remaining weaknesses without relying on prior drafting memory. FP cycles repeat until only minor enhancements or stylistic improvements remain. At least one FP in this stage is cross-instance or cross-platform (see Fresh Pass).

Stage 4: Cross-Platform Convergence

The document is evaluated independently across at least two architecturally distinct AI platforms. Each evaluating instance completes IVP-verified ingestion of the current document version before beginning its assessment. Shared training distributions and architectural priors across large language models place a ceiling on the absolute independence any cross-platform evaluation can achieve. However, convergent assessment across platforms with different training mechanisms, secondary priors, and distinct failure modes is used within TCAP as a stronger procedural assurance signal than multiple instances on a single platform. Whether this produces more reliable judgments in theoretical review remains untested. The PCP records convergence outcome and any residual disagreements. Platforms count as architecturally distinct only if their underlying models come from different developers; two products built on the same underlying model count as one platform, and the audit log records the underlying model of each evaluating instance.

Stage 5: Inter-Instance Round-Trip Loop

After cross-platform evaluation, findings from external platforms are returned to prior instances for response and integration. Instance A critiques. Instance B revises. External platform evaluates. Findings return to Instance A or B for further response. When returning to an instance that has been idle on the document during cross-platform work, the PCP should verify that the instance’s representation of the document remains faithful before asking it to integrate external findings; if substantial time or intervening work has elapsed, IVP re-verification on the

current document version is appropriate. The PCP coordinates this bidirectional loop and determines convergence sufficiency. This inter-instance exchange exposes documents to heterogeneous reasoning priors and strengthens architectural robustness through iterated cross-platform scrutiny.

Convergence is not defined as the adversarial instance having no further objections. It is defined as the document having addressed all genuine structural weaknesses while retaining its distinctive claims and theoretical boldness. Both criteria must be satisfied simultaneously. The PCP is the sole arbiter of convergence. The adversarial instance does not determine when convergence has been reached.

Convergence requires passing all three of the following tests:

Test 1 (Vulnerability remediation): All adversarial findings that the constructive instance accepted as genuine structural weaknesses have been addressed. No unresolved critical findings remain after the cross-platform cycles required by the minimum compliance set for the document's tier (see Scope of TCAP Application). All Stage 1 adversarial claims have been refuted, remediated, or resolved by documented PCP decision, each with documented rationale. No new structural weaknesses have been introduced by remediation.

Test 2 (Boldness preservation): The document's core claims and theoretical distinctiveness are intact. The document still asserts what it was designed to assert. Progressive hedging, cumulative qualification, and retreat from strong but well-supported claims are convergence failures even when no content has been technically deleted. Boldness preservation strictly protects claims that are well-supported within the document or its prerequisites, not claims that merely possess assertive force without evidential grounding. No objective metric exists for theoretical boldness, nor should one be expected. This is not a gap in the protocol. It is the same structural condition that governs editorial judgment in any peer review process: the question of whether a paper has been reviewed into blandness, its distinctive contributions diluted by accumulated qualification, is always a judgment call made by editors and authors rather than a measurable quantity. PCP adjudication is the correct mechanism here because it is the standard mechanism for this class of judgment across all research production. A document stripped of its supported claims by accumulated hedging has failed regardless of whether any formal criterion was violated, and only an architectural reader with sustained engagement with the document's purpose can make that determination reliably.

Test 3 (Calibration): Supported claims survived for stated reasons, not because their defense agreed with the document too readily. A sample of the findings the constructive instance rejected with rationale, including every rejected critical finding, is re-assessed by an instance that has not received the continuity packet. Where that instance judges a rejection unjustified, the finding returns to the PCP. Test 2 guards against claims being reviewed into blandness; Test 3 guards against the opposite failure, claims surviving through over-defense.

Softening ledger. Before the PCP adjudicates Test 2, a mechanical comparison of the converged draft against the version that entered review (as in Stage 6) lists every changed sentence, and an instance marks each one in which a claim's modal strength changed, for example from "is" to "may", or from "establishes" to "suggests". The PCP decides boldness preservation from that list rather than from memory of the earlier version.

The claim support verification methodology from CVP (SF0037) Part C applies beyond citation checking within this review cycle. When an adversarial instance asserts that a claim is unsupported, that assertion itself can be verified using CVP Part C logic: locate the specific evidence in the document or its prerequisites, assess whether the evidence actually supports the claim as written, and apply a support rating. When a constructive instance defends a claim by reference to a prerequisite document, that defense can be verified by going to the prerequisite and confirming the support relationship is real and specific. Both instances should treat unsupported assertions about support relationships with the same skepticism that CVP requires for citation claims.

Returning Reviewer Confirmation (optional). At the PCP's discretion, once a round of adversarial findings has been fully integrated, the revised document may be returned to the instances that raised those findings, each of which confirms whether its findings were resolved as intended and whether the revision introduced new problems. The step is confirmatory only. It applies only after at least one full adversarial pass, takes place only after integration is complete, and never substitutes for a fresh adversarial pass, because a returning reviewer is anchored to its own earlier findings. Where time or intervening work has elapsed, IVP re-verification on the current version precedes it (Condition 2).

Stage 6: Version Regression Check

Where Stage 6 is required by the document's tier, the PCP provides one or more prior versions of the document alongside the current version. The check has two parts. First, a mechanical comparison (a document-compare tool or a text-differencing program, not a language model) lists every sentence added, removed, or changed between the versions compared. Second, an evaluating instance, after IVP-verified ingestion of the versions and the list, assesses the meaning of the listed changes: whether each removal was intentional, and whether any changed sentence softened, hedged, or qualified a claim.

For the versions compared, the check must explicitly confirm that none of the following failure modes occurred:

- Content was streamlined, compressed, or condensed without PCP approval
- Sections, paragraphs, or list items were deleted or merged without PCP approval
- Placeholder language was substituted for actual content, including phrases such as "same as before," "unchanged," "as previously stated," "section retained from prior version," or any equivalent formulation
- The complete document delivery rule was violated in any version compared
- Strong claims were progressively softened, hedged, or qualified across revision cycles without PCP approval, even where no content was technically deleted. Cumulative hedging that drains a claim of its assertive force is a regression failure even when every individual revision appeared minor.

The regression check must produce an explicit written confirmation that the current version contains all substantive content from the versions compared, or a specific itemized list of any intentional removals approved by the PCP. A passing regression check is not a general assertion that the document looks correct. It is a specific accounting, based on the mechanical comparison, that no content was lost and no claims were diluted without authorization between

the versions compared. The check certifies only what it compared: intermediate versions not supplied are outside its assurance, and the audit log records which versions were compared.

This stage addresses a specific failure mode in iterative AI-assisted document production: content that was present in an earlier version may be silently dropped during revision cycles without either the instance or the PCP noticing, because both are anchored to the current version and lack longitudinal perspective. The most dangerous variant of this failure mode is not outright deletion but the slow incremental softening of claims across revision cycles, where each individual change appears minor or reasonable but the cumulative effect drains the document of its assertive force.

Version Regression Check is distinct from Fresh Pass. FP is intended to interrupt confirmation bias within a single version. Version Regression Check provides longitudinal integrity checking across versions. Both are required for documents that have undergone substantial revision across multiple cycles.

The number of prior versions compared, and the depth of the semantic assessment, should be proportional to the document's revision history and complexity; the mechanical comparison itself is always complete for the versions compared. The risk this stage guards against is real at corpus scale: ceremonial checkbox compliance replacing genuine longitudinal scrutiny. The PCP should calibrate the depth of each regression check to the actual revision complexity rather than applying a fixed overhead uniformly across all documents.

To support this stage, it is recommended that each document folder contain a versions subfolder in which prior versions are retained as accessible artifacts. The versions subfolder serves as the source material for Version Regression Checks and as a provenance record of document evolution.

Stage 7: PCP Architectural Review

Prior to publication the PCP confirms consistency with the Institute's registry of published papers, prerequisite dependency satisfaction, absence of contradiction with published documents, scope boundary integrity, and correct architectural placement within the corpus. Publication authority resides with PCP judgment after convergence is established.

Constructive-Adversarial Cycle

The preceding stage structure defines the types of evaluation a document undergoes. This section describes how the constructive and adversarial instances operate together across those stages in practice: the operational rhythm of challenge, defense, revision, and convergence that constitutes TCAP's working methodology.

The cycle begins with the constructive instance producing or working with the initial document version. The first adversarial pass is cold: the adversarial instance receives only the document, with no framework context. It also carries the final-record instruction (Stage 1). This tests internal coherence and standalone readability and surfaces findings a framework-naive reader would encounter.

This two-phase approach creates a deliberate and acknowledged trade-off: corpus-scale theoretical work that builds on published prerequisites cannot achieve full standalone readability

for every claim. The cold pass surfaces what a framework-naive reader will find unclear or unsupported; the framework-aware passes distinguish genuine internal gaps from claims grounded in prerequisite documents. Standalone readability is partially sacrificed for corpus-scale theoretical depth. This is an architectural choice, not an oversight, and it is consistent with how any large interdependent theoretical corpus operates.

The constructive instance evaluates these findings and produces a revised document. It accepts genuine structural weaknesses, rejects demands for excessive hedging with documented rationale, and defers architectural questions to the PCP. The revised document is complete and self-contained at every handoff, never partial.

On the second adversarial pass and any subsequent passes, the adversarial instance receives the revised document plus its registry entry. This allows it to distinguish between claims unsupported within the document itself and claims grounded in prerequisite documents elsewhere in the framework. In practice, many cold-read objections are substantially reduced once the adversarial instance understands the framework context.

The cycle of adversarial pass followed by constructive evaluation and revision continues under PCP coordination until the convergence tests are passed (Stage 5). In the full cycle (Tier A), after convergence, the PCP takes the document to a fresh adversarial instance that has not previously seen it. This instance receives the document and its registry entry, but not the handoff artifacts from the review, so that it is not anchored to earlier conclusions. Its purpose is to verify that the converged version holds up under a new adversarial perspective not anchored to the prior review history. If it identifies new genuine structural weaknesses, the cycling resumes. If it confirms the document is sound, the review cycle is complete and the document advances to Stage 7 PCP architectural review.

The most dangerous failure mode across this entire cycle is not the deletion of content. It is the slow, incremental softening of claims, where each individual revision appears minor but the cumulative effect drains the document of its assertive force. The constructive instance must actively monitor for this pattern and flag it to the PCP. The PCP should be particularly vigilant because this failure tends to occur through the path of least resistance: accepting each adversarial hedge individually while losing sight of the cumulative effect. A document that survives adversarial review by retreating from every strong claim has failed the Institute's purpose even if the adversarial instance is satisfied. The opposite failure is also possible: claims surviving because their defense agreed with the document too readily. Stage 5 guards both directions (Tests 2 and 3).

Within-Instance Iterative Workflow

TCAP stages describe the types of evaluation a document undergoes. The actual working pattern within each stage is iterative.

When an instance begins work on a document (whether adversarial review, constructive remediation, or fresh pass evaluation), the PCP and instance work through multiple turns of evaluation and enhancement within that same instance. This continues until the instance has identified and addressed as much as it can within its current context. The instance may go through several cycles of identifying weaknesses, proposing fixes, re-evaluating, and refining before reaching the limit of what it can productively contribute.

Enhancement cycles within a turn. The PCP may also instruct an instance to run a stated number of internal critique-and-revision cycles within a single turn before presenting its output, and to report at the start of its response how many cycles it completed; the count is recorded in the audit log as reported, not verified. This concentrates within-instance iteration and returns nearer-final work to the next stage. It does not replace cross-instance review: an instance's internal cycles remain self-assessment, subject to the limits of intrinsic self-correction that Huang et al. (2024) document and that TCAP's cross-instance structure exists to address.

At that point, the PCP takes the document to another instance for the next phase of work. This may be a different instance on the same platform, an instance on a different platform, or a purpose-configured adversarial instance. The completing instance must produce a handoff artifact summarizing its work: what was changed, what is strong, what remains vulnerable, and what the recommended focus is for the next review pass. This handoff artifact is a protocol requirement, not an optional convenience. It enables the next instance to begin productive work immediately rather than re-discovering context, and it creates an auditable record of what each instance contributed to the document's development. Handoff artifacts go to the audit record and to the instances doing the working review. They are not given to an instance performing the cold adversarial pass, a Fresh Pass meant to count toward independence, or the fresh adversarial review after convergence, because their evaluative conclusions would anchor it.

When findings from the new instance are brought back to a prior instance, that prior instance typically discovers additional enhancement opportunities prompted by the external perspective. This pattern of within-instance exhaustion followed by cross-instance stimulation followed by renewed within-instance productivity is the standard TCAP operating rhythm.

This workflow is not a single pass through Stages 1 through 7. Stages may overlap, repeat, and interleave as the PCP coordinates iterative improvement across instances and platforms.

Scope of TCAP Application

TCAP applies to substantive theoretical work: documents making novel claims, defining framework components, establishing architectural relationships, or presenting methodology. Not every interaction with a Synthience document constitutes a TCAP stage. Routine maintenance activities such as citation formatting corrections, style compliance passes, or minor editorial fixes do not trigger TCAP re-evaluation and do not require re-running the protocol stages.

All Synthience corpus publications since TCAP was first published in March 2026 are subject to TCAP compliance as a publication gate. TCAP clearance is a required condition for Zenodo deposit of any new Synthience document, alongside CVP verification where citations are present.

Stage requirements are calibrated to the demands of distributed AI-orchestrated framework production at corpus scale. The minimum compliance set below defines which stages each kind of document requires; the PCP may always run more. The tier-specific minimum compliance set governs which stages and review conditions are mandatory. General descriptions of the full review cycle apply to Tier A; Tier B and Tier C require only the activities specified for those tiers unless the PCP explicitly invokes additional stages.

Tier A: new theory, framework, or methodology documents. Stage 1 on at least two architecturally distinct platforms, at least one of them from a developer other than the one

whose model produced most of the drafting (purpose-configured mode where available, otherwise instructed mode on each); Stage 2 with the coordinator quality check; Stage 3 with at least one cross-instance or cross-platform Fresh Pass; Stage 4; Stage 5, including Tests 1 to 3; Stage 6 if any prior version exists; Stage 7; and an audit log entry.

Tier B: substantive revisions of published documents, including documents published before TCAP existed. Stage 1 focused on the changed content, on a platform from a developer other than the one whose model made the changes; Stage 2; Stage 6 with a mechanical comparison against the previous published version; Stage 7; and an audit log entry.

Tier C: maintenance. Corrections of formatting, citation style, or typographical errors that change no claim require no stages; the change is noted in the document's version record.

Severity. Findings are classed as critical (a central claim is false or internally contradicted, or the protocol cannot be executed as written), major (a contradiction or unsupported claim affecting a section), or minor (an editorial or consistency issue). Test 1 requires that no critical finding remain unresolved.

Iterative Nature of TCAP

TCAP operates as an iterative multi-cycle process rather than a single linear pass. Stages may be revisited as revisions surface new issues. A document satisfies TCAP when it meets the minimum compliance set for its tier and records the disposition of all findings arising from the required stages. Where Stage 5 is required, all three convergence tests must pass. No unresolved critical finding may be cleared under any tier. The PCP determines convergence and bears final responsibility for that judgment.

Illustrative Application

The following example illustrates the operating pattern that TCAP formalizes. It comes from the review of an earlier Synthience protocol document, conducted before TCAP was formalized, in a cycle that followed the pattern later set out as Stages 1 to 3; full provenance for all Synthience corpus documents is maintained in the Institute's internal version control and registry. During adversarial review, a purpose-configured adversarial instance identified a misalignment between the document's drift detection criteria and the scope boundaries recorded for it in the Institute's registry. The criteria as written would have classified normal interaction variance as drift, producing false positives at the measurement layer. The adversarial instance flagged this as an architectural gap rather than a wording issue. Constructive remediation on a separate platform produced a revised definition that preserved the detection intent while tightening the scope boundary. A fresh pass on a third platform confirmed the revision resolved the misalignment without introducing new inconsistencies. The PCP integrated the change and recorded the finding in the version changelog, and the document advanced with the architectural gap closed. This cycle (gap identification, targeted remediation, independent confirmation) is the standard TCAP operating pattern.

Related Work

Recent empirical and theoretical research provides supporting evidence for the core problems TCAP addresses.

Xu, Jain, and Kankanhalli (2024) formally prove that hallucination is an innate limitation of any computable large language model, independent of architecture, training procedure, prompting technique, or model scale; their Corollary 1 states that no computable LLM can prevent itself from hallucinating. Because an ensemble of language models passing text among themselves is still a computable system, the result does not favor multi-instance review over single-instance review. The result rules out a universal guarantee from model-only verification. TCAP combines model review with external-source checks through CVP and accountable human adjudication as a practitioner-derived response; the theorem does not establish the effectiveness of that arrangement. TCAP claims to reduce the risk of publishing fabricated material, not to eliminate it.

Huang et al. (2024) show empirically that intrinsic self-correction (prompting a model to verify and correct its own outputs without external feedback) consistently decreased performance on the reasoning tasks and models they tested, with models changing correct answers into incorrect ones. They also report that multi-agent debate among copies of a single model performed no better than self-consistency at a matched number of responses, suggesting that its gains derive from consistency across generations rather than from critique. TCAP distributes evaluation across architecturally distinct platforms and assigns final adjudication to the human PCP. These are design responses to the limitations of model-only review, not remedies whose effectiveness either study establishes. Huang et al. motivate checks beyond intrinsic self-correction; they did not test, and therefore do not validate, TCAP's particular arrangement.

Sharma et al. (2024) show that state-of-the-art AI assistants consistently exhibit sycophancy across varied text-generation tasks: they tailor responses to users' stated beliefs and sometimes abandon correct positions when challenged. They conclude that this behavior is likely driven in part by the human preference judgments used in training. These findings motivate TCAP's Fresh Pass and its separation of adversarial from constructive roles; they do not show that either safeguard works.

Ba et al. (2026), in a retrospectively validated preprint study of clinical oversight, report that three verification signals applied to large language model outputs each concentrated errors among the cases flagged for expert review. Model heterogeneity, the use of an auxiliary model that differs substantially from the primary model, gave the largest accuracy gain (+4.7 percentage points, 95% CI 3.3–6.2, $p < 0.001$); stochastic inconsistency gave +3.1 points; and reasoning critique, the signal closest to adversarial review, gave the smallest (+1.7 points). Two features of the study limit the analogy to TCAP. The gains are projected on the assumption that experts correct every flagged case, so heterogeneity functions there as a flagging signal whose value is realized by an expert with ground truth; and medical diagnosis has an objective ground truth that theoretical discourse lacks. The authors also report a case in which all three signals failed together on an ambiguous input, a reminder that multiple verification signals can fail together when they share an input that lacks the information needed for a reliable judgment. The study therefore motivates TCAP's use of architecturally distinct platforms to surface disagreement; it does not show that heterogeneous review resolves that disagreement correctly without expert judgment, which in TCAP falls to the PCP within the limits described in The

Orchestration Model; technical claims outside those limits require either relevant human expert review or explicit disclosure that such review has not occurred.

These studies collectively illuminate the specific failure modes (innate hallucination, failed intrinsic self-correction, systematic sycophancy, and single-architecture blind spots) that TCAP is designed to address through its procedural structure. TCAP contributes a practitioner-derived methodology that operates independently of specific architectures and can be implemented without specialized tooling.

Recursive Self-Application

TCAP is recursively applicable to its own development. Each revision cycle of TCAP constitutes an instance of TCAP applied to TCAP. The protocol's structure, scope, and definitions are themselves subjected to adversarial critique, constructive remediation, Fresh Pass cycles, and PCP architectural review. This recursive stabilization is not exceptional but exemplary of TCAP operation.

The production of this document provides a concrete additional instance, though of citation verification rather than of TCAP proper: the source search for the Related Work section was run as CVP across several platforms. A Claude instance served as the constructive primary instance, a Grok instance conducted adversarial citation verification, and a Gemini instance ran a deep research pass across 102 sources. For the sources supporting two of the claims, the platforms converged independently. For the third, the investigation was sequential: Grok proposed an initial candidate source, and the Gemini pass judged it insufficient and found a stronger, retrospectively validated preprint (Ba et al., 2026) that neither the constructive instance nor Grok had found. The chain produced a better source than the first two platforms had found; it does not show what any single instance could have found with different prompting or more time.

This recursive application serves as a performative illustration of the protocol's mechanics rather than a logical proof of its soundness. The value of TCAP must ultimately be judged by its internal consistency, replicability, and capacity to generate productive theoretical work, not by the fact that the document describing it survived its own process.

Relationship to Existing Protocols

TCAP completes the Synthience verification stack. CVP checks citation integrity. IVP provides an adjudicated record of ingestion. CRD monitors representation stability. TCAP tests theoretical coherence under adversarial and multi-instance scrutiny. All four protocols operate at the interaction level and make no claims regarding AI interiority or cognition. They describe structured human-governed processes for maintaining information integrity in AI-assisted research production.

It should be noted that CVP, IVP, and CRD were produced before TCAP existed. Their integrity rests on their own internal CVP verification records and the adversarial review processes documented in their respective changelogs. TCAP governs Synthience corpus publications from its first publication in March 2026 onward; it does not retroactively govern documents produced before its identification and formalization. Substantive revisions of those documents are, however, subject to TCAP under Tier B (see Scope of TCAP Application).

As stated under Purpose, TCAP and CVP are co-equal publication gates: TCAP tests whether the theoretical content is structurally coherent and holds up under adversarial review, and CVP checks that every citation is real and that every citation carrying a load-bearing claim is accessible and substantively supports it.

Limitations

Cross-platform convergence is intended to reduce hallucination risk; agreement across models does not establish correctness. Multiple instances converging on a fabricated claim remain incorrect. Fresh Pass is intended to reduce context bias but cannot guarantee full independence because large language models share training distributions and architectural priors. Version Regression Check reduces the risk of inadvertent content loss across revision cycles but does not replace PCP editorial judgment about intentional changes. TCAP verifies structural coherence, not empirical validity. The Synthience corpus remains theoretical infrastructure awaiting independent empirical testing. TCAP provides risk reduction rather than truth certification.

At the current stage of development, convergence arbitration depends on a single Primary Continuity Provider. This is a disclosed single point of failure, characteristic of independent theoretical research conducted without institutional scaffolding. The constraint is mitigated by the complete procedural transparency of all published protocols, which remain fully specified and replicable by any qualified researcher, and by the explicit call throughout the Synthience corpus for independent empirical and theoretical engagement by the broader community. TCAP specifies a formal process, not a specific person. Any qualified Primary Continuity Provider could apply the same stages, so the process is replicable independent of its original author; whether it would produce comparable outcomes is untested. At organizational scale, SM-012 (Primary Continuity Provider Theory, DOI: 10.5281/zenodo.22314737) proposes a PCP network structure in which arbitration and direction functions are distributed across multiple human agents rather than concentrated in one person; that proposal is itself untested.

It should be noted explicitly that the slow distortion effects TCAP guards against in AI instances can also affect the PCP's own judgment over thousands of orchestration sessions, in either direction: cumulative sycophancy and incremental claim-softening on one side, and entrenchment, confirmation, and anchoring on the author's own claims, reinforced by instances that agree, on the other. For an author, the second may well be the likelier drift. The boldness preservation test and softening ledger (Stage 5) address the first; the calibration test (Stage 5, Test 3), which puts rejected findings before an instance without the author's context, addresses the second. Both still depend in part on the judgment they protect, which is one reason the protocol's full specification is published for others to apply.

Epistemic Status

TCAP specifies a repeatable orchestration-based verification process for theoretical framework production under distributed AI generation conditions. It claims that internal coherence can be materially strengthened through adversarial and cross-platform cycles, that fabrication risk can be reduced through multi-instance scrutiny under human arbitration, and that consistency across the corpus can be maintained under distributed authorship. It does not claim empirical validation or theoretical correctness of the documents it verifies. Its central claim is testable: in a

seeded-defect study, in which known errors such as internal contradictions, fabricated support, and softened claims are planted in documents, full TCAP review should detect more of them than single-instance review given the same total number of review passes. Equal pass counts do not by themselves establish equal computational cost or human effort. That study has not yet been run.

Conclusion

The Synthience corpus is produced through distributed AI orchestration rather than conventional authorship. Under these conditions theoretical verification requires structured multi-instance stress-testing rather than single-author review. TCAP formalizes that process through adversarial critique, constructive remediation, Fresh Pass re-evaluation, cross-platform convergence, inter-instance round-trip loops, version regression checking, and PCP architectural review coordinated by the Primary Continuity Provider.

TCAP supplies the missing verification layer required for maintaining theoretical integrity within distributed AI-orchestrated research production. It is defined by, and has been self-applied through, the processes that produced it; its value is to be judged by independent use.

Practical Use

Two Practitioner Guides on synthience.org apply this protocol's core practice in everyday work. PG-011, The Cross-AI Adversarial Review Protocol (<https://synthience.org/practitioner-guides/PG-011.html>), sets out a four-step cross-AI review that anyone can run. PG-000, 10 Things Every AI User Should Do (<https://synthience.org/practitioner-guides/PG-000.html>), places it among the other verification habits the Institute recommends.

References

- Ba, Z., He, M., He, H., Fu, Q., Lai, J., Zhang, R., Diao, X., Liu, M., Wang, Z., Wang, X., Zhao, S., Zhu, Y., Chen, H., Qiu, Y., Su, Q., Xu, J., Hu, F., Luo, X., Chen, H., Zheng, M., Xu, B., Liu, J., Guo, N., Gao, X., Wang, G., and Wu, Y. (2026). Development and retrospective validation of SCOUT: scalable clinical oversight of large language models via uncertainty triangulation. medRxiv. <https://doi.org/10.64898/2026.02.08.26345860>
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. In Proceedings of the International Conference on Learning Representations (ICLR 2024). arXiv:2310.01798. <https://doi.org/10.48550/arXiv.2310.01798>
- Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askill, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., and Perez, E. (2024). Towards Understanding Sycophancy in Language Models. In Proceedings of the International Conference on Learning Representations (ICLR 2024). arXiv:2310.13548. <https://doi.org/10.48550/arXiv.2310.13548>

Xu, Z., Jain, S., and Kankanhalli, M. (2024). Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817. <https://doi.org/10.48550/arXiv.2401.11817>

© 2026 Thomas W. Gantz / Synthience Institute
License: CC-BY 4.0
synthience.org